



Government
Digital Service



Department for
Science, Innovation
& Technology

Secure coding practices, Static and Dynamic Application Security Testing (SAST and DAST): Alignment with Secure by Design

Contents

Background	3
Purpose	3
1. How secure coding practices align with Secure by Design	4
1.1 Principles of Secure by Design	4
1.2 Secure by Design activities	5
1.3 Aligning secure coding practice guidelines with Secure by Design principles	5
1.4 Secure coding and Secure by Design activities	6
2. How software composition analysis (SCA) aligns with Secure by Design	7
2.1 SCA alignment with Secure by Design principles	7
2.2 SCA alignment with Secure by Design activities	9
2.3 SCA implementation pattern	10
3. Static Application Security Testing (SAST) and Secure by Design	11
3.1 SAST patterns	12
4. Dynamic Application Security Testing (DAST) and Secure by Design	15
4.1 DAST patterns	15
5. Integrating SAST and DAST into the software development life cycle (SDLC)	18
6. More information	20

Background

Secure coding practices are sets of guidelines and techniques developers follow to write software that are resilient to security threats and vulnerabilities. These practices aim to reduce the likelihood of issues such as injection attacks, buffer overflows, weak authentication and insecure data management.

Strategies include:

- input validation and sanitisation
- enforcing robust authentication and access controls
- encrypting sensitive information both in transit and at rest
- secure error handling that avoids revealing system internals
- steering clear of outdated or vulnerable libraries

Secure coding also helps us keep to design principles like least privilege and defense in depth, keeping dependencies up to date, and using tools such as linters and static code analysers to identify problems early. Integrating these practices throughout the development life cycle helps organisations mitigate security risks, safeguard user data and maintain compliance with security regulations like the UK General Data Protection Regulation (UK GDPR) and the Data Protection Act 2018.

The government [Secure By Design](#) approach helps delivery teams and security professionals incorporate effective cyber security practices when building digital services and technical infrastructure. This approach provides risk-driven activities for building appropriate and proportionate cyber security controls within digital services.

Purpose

The purpose of this document is to outline secure coding practices, tools and techniques and align them with the Secure By Design approach.

More about Secure by Design

- [Secure by Design policy](#)
- [Implementing Secure by Design](#)
- [Guide to adopting Secure by Design](#)

1. How secure coding practices align with Secure by Design

Secure coding practices are fundamental to realising the vision of Secure by Design, where security is integrated seamlessly throughout the software development life cycle rather than being an afterthought. Developers can create software that is inherently resilient against evolving threats by embedding rigorous security measures into the code. These might include:

- early threat assessment
- strict access controls
- layered defenses
- secure defaults
- robust input validation

The most effective way to apply Secure by Design in software engineering is to directly connect coding practices to the principles. This connection creates a bridge between policy and daily development tasks.

For example, Secure by Design principle 2, Source secure technology products, relates closely to how developers manage external dependencies. Choosing reputable libraries, verifying package signatures and using tools to scan for vulnerabilities are all coding practices that operationalise this principle.

Similarly, principle 10, Make changes securely, is reflected in structured workflows like code reviews, automated builds and tightly controlled deployment pipelines.

By making these links explicit, teams gain clarity and accountability. Product owners and managers can see how specific coding behaviours support organisational objectives, while developers understand why particular practices are non-negotiable. For example, strict input validation enforces both minimised attack surfaces and layered defences, while implementing strong logging practices helps detect and respond.

1.1 Principles of Secure by Design

The [10 principles of Secure by Design](#) are high-level commitments that guide how resilient systems should be created and maintained. Each is intended to reduce risk by shaping the way organisations approach design and delivery.

For developers, these principles translate directly into coding choices and architectural patterns. For example, minimising the attack surface (principle 7) means stripping away unnecessary features, eliminating unused code and tightening access controls so fewer pathways exist for attackers to exploit. Likewise, defending in depth (principle 8) involves layering controls so that even if one safeguard fails, others remain in place to block or slow down an attack.

These principles are deliberately broad so they can be applied across technologies, environments and maturity levels. They act as a compass, not a checklist. A developer writing a web application may apply them differently to a team building embedded systems, yet the outcomes should be the same: services that are secure, resilient, and maintainable.

For example, embedding continuous assurance (principle 9) could involve automated testing pipelines in one context and rigorous manual review cycles in another. The real value lies in understanding how each principle connects to coding practices, turning abstract security goals into concrete technical actions.

1.2 Secure by Design activities

Secure by Design offers [21 activities](#) that help meet the principles. These activities break down what government organisations should actually do during different phases of the service life cycle.

For example, conducting threat modelling helps teams identify where vulnerabilities may emerge, while implementing structured vulnerability management makes sure that discovered weaknesses are tracked and fixed quickly.

These activities are not one-time tasks. They are ongoing responsibilities that must evolve alongside the service itself.

For development teams, many of these activities map directly to everyday work. For example:

- preparing a secure service needs coding standards to be set and integrated into design reviews
- managing risks means understanding how third-party components or libraries might introduce flaws
- anticipating vulnerabilities often involves writing tests designed to mimic real-world attack techniques

By embedding these activities into normal workflows, coding teams help ensure that services align with government expectations while actively reducing the likelihood of compromise.

1.3 Aligning secure coding practice guidelines with Secure by Design principles

Each Secure by Design principle can guide and shape how different coding practices are applied.

For example, principle 4, Design usable security controls, highlights the need for developers to design mechanisms that protect users without overwhelming them. At a technical level, this means building authentication and authorisation systems that are both robust and user-friendly. Multi-factor authentication (MFA) should be put in place where

possible, but implemented in a way that does not frustrate or confuse legitimate users. Secure password reset processes should use short-lived tokens and safe communication channels, with logic in the code that blocks replay attempts and invalidates used tokens.

Principle 6, Design flexible architectures, calls for designs that can evolve as threats and requirements change. Developers can achieve this by:

- writing modular code
- using microservices
- enforcing strong boundaries between components

For example, a compromised service should not automatically provide attackers with access to the wider system. Technically, this requires building APIs with strict validation, making sure each service authenticates its peers and isolating data so that permissions are enforced at multiple layers. Writing code that avoids tight coupling makes sure that you can integrate patches, upgrades or new controls without destabilising the system.

1.4 Secure coding and Secure by Design activities

Every Secure by Design activity has clear implications for developers and coding practices.

For example, the requirement to access the security impact of system changes means that no update should be treated as routine. Pull requests (PRs) should trigger both automated security checks and structured peer reviews. Developers reviewing code must examine not only whether features work but also whether security rules are being followed.

This could mean confirming that new API endpoints validate inputs correctly, that authorisation is applied consistently or that error handling does not reveal sensitive information. Code should only be integrated into the main branch when these sorts of checks are passed.

Similarly, the activity around retiring service components securely is often overlooked but critical. Old modules, APIs or configuration files must be removed cleanly rather than simply being left dormant. Even unused code can provide a backdoor if attackers discover it.

A secure retirement process includes deleting obsolete endpoints, revoking any associated credentials, and ensuring logs or secrets tied to that component are securely wiped. Developers have a direct role in writing the code that decommissions these elements safely, preventing forgotten functions from lingering as hidden risks.

Related guidance

- [Secure development and deployment guidance \(NCSC\)](#)
- [Software Security Code of Practice \(GOV.UK\)](#)

2. How software composition analysis (SCA) aligns with Secure by Design

Software composition analysis (SCA) is an automated, repeatable capability. It finds, inventories and assesses all third-party and open-source components in software, as well as their transitive dependencies, associated vulnerabilities and often their licensing.

SCA produces machine-readable inventories (software bills of materials or SBOMs), vulnerability alerts and risk scoring that directly support many Secure by Design principles and activities. In particular, this includes:

- sourcing secure technology products
- discovering and responding to vulnerabilities
- embedding continuous assurance
- minimising the attack surface

Implemented across the continuous integration and continuous delivery (CI/CD) pipeline and the organisation's vulnerability-management processes, SCA is a foundational control for software supply-chain security.

Beyond technical scanning, SCA also creates accountability and transparency across organisations. It gives leadership, regulators and auditors verifiable evidence that open-source risk is being actively monitored and mitigated. This is increasingly vital as regulators, including UK government buyers, expect visibility from suppliers' software supply chains. By aligning SCA outputs with Secure by Design principles, organisations can present an assurance narrative that's coherent.

SCA also supports strategic planning and resilience. By understanding dependency relationships at scale, organisations can anticipate systemic risks such as a critical library affecting multiple services or products. This visibility supports enterprise-wide patch prioritisation, investment decisions, and supplier engagement. This means SCA does not just contribute to individual project security, but also to overall organisational cyber resilience.

2.1 SCA alignment with Secure by Design principles

Principle 1: Create responsibility for cyber security risk

Alignment happens when SCA outputs (SBOM, dashboards, monthly risk register entries) give risk owners concrete evidence to accept, mitigate or escalate third-party component risk.

To achieve this, you should:

- designate an SCA owner
- include SCA metrics in leadership reports

Principle 2: Source secure technology products

Alignment happens when:

- SCA helps perform due diligence on vendor-supplied components and track third-party product vulnerabilities
- SBOMs and vulnerability posture are a procurement requirement

To achieve this, you should:

- require SBOMs and SCA scans from suppliers
- include SCA checks in supplier acceptance criteria

Principle 3: Adopt a risk-driven approach

Alignment happens when SCA enables risk-driven prioritisation (exploitability, exposure, business criticality, transitive depth).

To achieve this, you should enrich SCA findings with asset criticality so risk owners can prioritise.

Principle 4: Design usable security controls

Alignment happens when:

- SCA is integrated into developer workflows (PR checks, CI alerts) so remediation is fast and low-friction
- there are clear, actionable remediation steps

To achieve this, you should:

- fail builds only for high-policy breaches
- provide automated PR fixes where possible

Principle 5: Build in detect and respond security

Alignment happens when SCA provides continuous detection of new vulnerabilities in dependencies and integrates with incident or vulnerability management (ticketing, patch playbooks).

To achieve this, you should wire SCA alerts to the vulnerability management process and SOC runbooks.

Principle 6: Design flexible architectures

Alignment happens when modularisation and replaceability is encouraged because dependencies surfaced by SCA make it easier to swap or isolate risky components.

To achieve this, you should use SCA reports in architectural reviews to reduce coupling to risky libraries.

Principle 7: Minimise the attack surface

Alignment happens when SCA shows unused or ancillary dependencies and transitive components that can be removed or sandboxed, reducing exposed code.

To achieve this, you should include dependency pruning in the backlog and deploy minimal runtime images.

Principle 8: Defend in depth

Alignment happens when:

- component risk is one layer
- SCA is combined with runtime protections (Web Application Firewall (WAF), Runtime Application Self-Protection (RASP))
- builds and access controls are signed and verified to increase resilience

To achieve this, you should use SCA as an input to layered mitigations.

Principle 9: Embed continuous assurance

Alignment happens when SCA is inherently continuous, so scheduled scans, CI gating and SBOM generation provide ongoing assurance and audit trails.

To achieve this, you should make SCA reports part of release evidence.

Principle 10: Make changes securely

Alignment happens when SCA runs on every change (branch builds, PRs, image builds) so the security impact of new or updated dependencies is known before deployment.

To achieve this, you should require SCA pass or risk acceptance prior to release.

2.2 SCA alignment with Secure by Design Activities

Each of the 21 Secure by Design activities can benefit from SCA outputs. From the early planning stages to service retirement, SBOMs and vulnerability reports supply the evidence needed for informed risk management.

For example, during business case development, expected costs for SCA should be factored into planning. In supplier management, SBOMs confirm that third-party software meets security expectations. And in vulnerability management, SCA alerts can be linked directly into ticketing systems for remediation.

The reusability of SCA evidence is especially powerful. A single SBOM can support asset registers, risk assessments, compliance checks and decommissioning activities. This efficiency aligns with Secure by Design's emphasis on proportional, non-burdensome security controls.

2.3 SCA implementation pattern

A phased approach helps government organisations adopt SCA effectively. Basic implementation might involve periodic scans and manual reporting. Mature adoption integrates scanning into CI/CD pipelines, automatically generating SBOMs, enforcing policy thresholds and creating remediation tickets.

Scaling SCA across large organisations needs balance. Central teams can set policies, formats and thresholds, while individual engineering teams adopt tools suited to their technology stacks. This hybrid model maintains consistency without stifling flexibility, and supports Secure by Design's principles of adaptable, risk-aware security.

Related guidance

- [SBOMs and the importance of inventory \(NCSC\)](#)
- [Supply chain: Guidance \(NCSC\)](#)

3. Static Application Security Testing (SAST) and Secure by Design

Static Application Security Testing (SAST) is a white-box security analysis technique that evaluates source code, bytecode or binaries without executing the programme. By scanning through a codebase, it identifies insecure coding patterns, logic flaws and misconfigurations that may otherwise remain invisible until an application is deployed.

This form of testing is designed to help developers address vulnerabilities at the earliest stages of the software development life cycle.

Typical weaknesses uncovered by SAST include:

- injection flaws
- buffer overflows
- insecure handling of input or output
- unsafe deserialisation
- misuse of cryptographic functions

Unlike dynamic testing, which needs a running environment, SAST integrates directly into the development process. It can be:

- applied in local development environments
- applied within integrated development environments (IDEs)
- embedded into build pipelines

This flexibility lets government organisations adopt a ‘shift-left’ strategy, embedding security checks where they are cheapest to fix and easiest to remediate.

SAST results are typically immediate and actionable, providing developers with line-by-line insights into vulnerable sections of code.

Another defining feature of SAST is that it can enforce secure coding standards across teams. By embedding security rules and compliance policies into automated scans, organisations can make sure their development practices meet internal policies and external regulations. Over time, this fosters a culture of security awareness within engineering teams and reduces reliance on after-the-fact remediation.

SAST enables:

- early identification of vulnerabilities like SQL injection, XSS and insecure APIs during coding
- real-time feedback to developers through an integrated development environment (IDE)
- automation of scans during code commits or PRs in CI/CD pipelines
- enforcement of secure coding policies across teams
- developer education using contextual remediation guidance and links to secure coding standards

Since SAST catches issues before code is compiled or deployed, it prevents insecure practices being embedded in the application and shortens remediation time.

3.1 SAST patterns

SAST is a method of scanning application source code or compiled code for known vulnerabilities without executing the application. It's designed to catch security flaws early, during the development life cycle and helps enforce secure coding practices.

These patterns outline how SAST can be practically applied to produce security controls that are:

- early
- consistent
- enforceable

3.2.1 IDE-level integration pattern

The most developer-centric pattern is integrating SAST tools directly into IDEs such as Visual Studio Code or Eclipse. When configured this way, SAST works as a real-time assistant, offering security insights as developers write or edit code.

This pattern lets developers detect vulnerabilities such as SQL injection, improper input validation or insecure cryptographic practices at the moment of creation, rather than waiting for a centralised scan. The feedback is contextual, helping the developer understand not just that something is wrong, but also why and how to fix it.

The IDE integration pattern supports the Secure by Design approach by enabling developers to self-remediate issues before the code leaves their local machine. It transforms security from a reactive burden to a natural part of the development workflow.

Secure by Design alignment

This pattern supports principles:

- 1. Create responsibility for cyber security risk
- 4. Design usable security controls
- 9. Embed continuous assurance

It aligns to the activities:

- [Considering security within the business case](#) (early responsibility and cost avoidance)
- [Understanding business objectives and user needs](#) (by making fixes developer-friendly)
- [Implementing a vulnerability management process](#) (feeds issues earlier into life cycle)

3.2.2 Pre-commit code screening pattern

This pattern introduces SAST scanning right before code is committed to the version control system, typically using Git hooks. Pre-commit hooks run local SAST scans automatically every time a developer attempts to commit changes.

If vulnerabilities of a predefined severity are detected, the commit is either blocked or flagged. This ensures insecure code is identified and addressed before it enters the shared repository, preventing vulnerabilities proliferating across branches or merges.

This pattern is particularly effective in high-velocity environments where code is pushed rapidly and frequently. It encourages good practice early in the process, keeping the shared codebase in a healthier state and reducing issues raised in later security reviews.

Secure by Design alignment

This pattern supports principles:

- 1. Create responsibility for cyber security risk
- 9. Embed continuous assurance
- 10. Make changes securely

It aligns to the activities:

- [Tracking Secure by Design progress](#) (measurable gates)
- [Implementing a vulnerability management process](#)
- [Evaluating the security impact of changes](#) (PR-level checks)

3.2.3 CI/CD Pipeline static analysis pattern

In this pattern, SAST is embedded into CI/CD pipelines, where every PR or merge triggers an automated static scan. These scans typically cover the entire codebase or the delta from previous commits, depending on how they're configured.

By including SAST as a mandatory step in the pipeline, organisations can make sure that every code change is vetted before being integrated or deployed. The results can be used to fail builds, trigger security alerts or update dashboards used by developers and security teams alike.

This pattern aligns well with DevSecOps methodologies and allows for scalable enforcement of security policies across teams and services. It also supports auditability and compliance by generating traceable scan logs.

3.2.4 Enforced security policy pattern (policy-as-code enforcement)

This approach takes CI-integrated SAST a step further by introducing formal enforcement policies. These policies dictate that builds or deployments are automatically halted if certain vulnerability thresholds are exceeded, such as the presence of high or critical severity issues.

By codifying security requirements into the pipeline, this pattern makes sure that security gates are binding and enforceable. It empowers security teams to define rules and lets developers see exactly what's required to move code forward.

Secure by Design alignment

This pattern supports principles:

- 2. Source secure technology products
- 10. Make changes securely

It aligns to the activities:

- [Tracking Secure by Design progress](#)
- [Managing third-party product security risks](#) (when policies codify allowed libraries/APIs)

3.2.5 Custom rule and business logic analysis pattern

Standard SAST rule sets catch general security issues, but every application has its own context, internal patterns and business logic. This pattern involves defining custom detection rules to address specific internal risks.

For example, an organisation may have custom authentication flows, unique API libraries or sensitive data handling conventions. With tools like CodeQL or Semgrep, security teams can write rules to enforce best practices that are specific to their objectives in government.

This approach increases the precision and relevance of scan results, significantly reducing false positives and enabling proactive identification of logic-specific vulnerabilities that generic rules might miss.

4. Dynamic Application Security Testing (DAST) and Secure by Design

Dynamic Application Security Testing (DAST) is a black-box testing method that examines a running application to detect vulnerabilities by simulating real-world attacks. It evaluates how the application behaves in a deployed state, without needing access to the source code. DAST complements SAST by catching flaws that emerge only at runtime.

DAST supports the Secure by Design approach by:

- testing for runtime issues like authentication bypass, insecure session handling and logic flaws
- identifying misconfigurations, missing headers and security control failures in staging or production environments
- validating that implemented security mechanisms function correctly under actual attack conditions
- assessing the full deployed stack, including web servers, middleware and APIs
- enhancing pre-release security assurance as part of integration, quality assurance (QA) or user acceptance testing (UAT) phases

By using DAST during the later stages of development and even post-deployment, organisations make sure that applications are secure not just in theory, but in practice as well.

4.1 DAST patterns

DAST tools operate at the opposite end of the spectrum from SAST. They scan running applications, simulating how an attacker might interact with the system. These tools identify vulnerabilities in runtime environments, configurations and exposed surfaces. DAST patterns describe how to strategically insert these tools into different stages of application deployment.

4.1.1 Staging environment runtime scanning pattern

This is one of the most common and effective DAST implementations. Before an application is pushed to production, it is deployed to a staging or test environment that mirrors the production stack. A DAST tool is then used to scan the live app.

The scanner navigates through the web user interface (UI), APIs, forms and endpoints. This uncovers issues like:

- cross-site scripting (XSS)
- SQL injection
- broken authentication
- insecure headers
- exposed error messages

This pattern adds value by testing the application in its assembled state, including the influence of runtime behavior, platform configurations and third-party components. It complements SAST by finding issues that are only apparent at execution time.

4.1.2 Pipeline-driven dynamic scanning pattern

Here, DAST is treated as a fully automated step in the CI/CD process. After the application is built and deployed in a test container or ephemeral environment, the DAST scan is triggered automatically as part of the pipeline.

This enables continuous security validation every time a deployment occurs. It ensures that even frequent or minor changes get scanned, providing a scalable model for security testing in DevOps-driven environments.

It's particularly powerful for microservices and containerised platforms where applications are deployed dynamically and at high frequency.

4.1.3 Authenticated dynamic testing pattern

Unauthenticated scans are limited to publicly exposed areas of an application. However, most security-critical functionality is hidden behind login walls. This is where authenticated scanning becomes essential.

In this pattern, DAST tools are configured to log in using test credentials or scripts, letting them access and test secure areas of the application such as user profiles, payment flows, admin dashboards or role-based functions.

This approach reveals vulnerabilities like:

- authorisation flaws
- insecure direct object references (IDOR)
- privilege escalation paths
- session mismanagement

All of these would be invisible to unauthenticated scans.

4.1.4 Recurring vulnerability monitoring pattern

Instead of integrating DAST into pipelines or relying solely on manual scans, this pattern involves scheduling automated scans at fixed intervals, for example, daily, weekly or monthly.

Scheduled scans provide an ongoing layer of proactive monitoring, useful for detecting vulnerabilities introduced by platform updates, environment drift, misconfigurations or changes to dependencies.

This pattern is often used in parallel with pipeline-based DAST to provide temporal coverage, helping teams make sure their environments remain secure even between releases.

4.2.5 Passive monitoring in live environments pattern

DAST doesn't always need to be active and intrusive. In highly sensitive production environments, passive scanning can be used to monitor traffic in real time without injecting test payloads.

This pattern involves placing DAST proxies or RASP-like (Runtime Application Self-Protection) agents in observation mode, inspecting requests and responses for signs of insecure behavior, anomalous headers or weak session configurations.

It's a low-risk way to gain visibility into live traffic and user behavior, and often serves as the final layer of defense in a mature security architecture.

Secure by Design alignment

This pattern supports principles:

- 5. Build in detect and respond securely
- 9. Embed continuous assurance

It aligns to the activities:

- [Assessing the effectiveness of security controls](#)
- [Implementing a vulnerability management process](#)
- [Managing observability](#)

5. Integrating SAST and DAST into the software development life cycle (SDLC)

SAST and DAST should be mapped carefully to different phases of the software development life cycle (SDLC) to support Secure by Design from end to end.

5.1 Planning and design

At the very start of the SDLC, security requirements should be defined alongside functional ones. This is when threat modelling takes place so that attack vectors and design mitigations can be anticipated.

Here, SAST plays a role in shaping secure coding standards by embedding specific rulesets into development practices, making sure the team writes code with security in mind.

While DAST is not typically applied at this stage, design-time considerations such as defining secure authentication flows, API boundaries and error handling strategies prepare the ground for meaningful runtime testing later.

5.2 Development

During active coding, SAST should be integrated directly into developers' workflows using IDE plug-ins, pre-commit hooks and continuous integration (CI) pipelines.

This allows insecure coding patterns, use of hardcoded secrets or risky library dependencies to be identified. They can then be corrected immediately, long before they can propagate into builds. Developers receive fast feedback on vulnerabilities, reinforcing Secure by Design practices.

DAST is not yet applied here, because the application is still being constructed and is not ready for dynamic testing.

5.3 Build and test

When code is compiled and deployed into test environments, both SAST and DAST play critical roles.

SAST continues running in the build pipeline, making sure no new vulnerabilities slip in as the application evolves.

In parallel, DAST is introduced to test the running application in a controlled staging environment. This includes probing for:

- misconfigurations
- insecure headers
- injection flaws
- broken authentication flows

Running SAST and DAST together in this phase provides both code-level and runtime-level assurance before the application progresses further.

5.4 Pre-release

Before deploying to production, a comprehensive DAST assessment should be conducted in a staging environment that mirrors the live system as closely as possible. These scans should be authenticated, meaning they simulate real user interactions and check for session handling issues, privilege escalation and access control weaknesses.

At the same time, SAST should be rerun to verify that all previously identified vulnerabilities have been addressed and that no new ones were introduced during late-stage changes.

This combined approach acts as a final quality gate for security before release.

5.5 Deployment

Once the application goes live, security testing does not stop. While SAST has less of a role in production, DAST should be integrated into deployment processes for continuous runtime monitoring.

Automated, non-intrusive DAST scans can identify emerging risks, like:

- misconfigurations
- exposed endpoints
- regressions introduced by deployment changes

These checks reinforce the principle of continuous assurance, making sure that what was secure in staging remains secure in production.

5.6 Maintenance

After deployment, applications inevitably evolve with patches, feature updates and dependency upgrades.

In this phase, both SAST and DAST must remain part of the maintenance life cycle. SAST ensures that new code merges adhere to secure coding practices, while DAST validates that deployed changes have not weakened runtime defenses.

Vulnerabilities that are discovered are tracked through issue management systems, verified upon remediation and re-tested with both SAST and DAST. This establishes a feedback loop that continuously strengthens security throughout the product's operational lifetime.

6. More information

NCSC guidance

- [Produce clean and maintainable code](#)
- [Secure development and deployment guidance](#)
- [Secure development principles](#)
- [Software Security Code of Practice: Implementation guidance](#)

GOV.UK guidance

[Cyber Governance Code of Practice](#)